Fully In-SPICE Analog Predictive-Coding Learning for Nonlinear 2D Classification: A Position Paper

Research log, `spicenn2`

June 9, 2026

Abstract

We attempt to train an analog circuit to classify three hard 2D datasets — *circles*, *rings*, *spirals* — to > 0.95 test accuracy on each, under a strict rule: **both inference and training run entirely inside SPICE** (weights are charge on capacitors; the weight update is performed by transistor multipliers; no behavioral elements), and the controller may only cycle the data. This paper reports an honest account of what works and what does not. The two most consequential findings are *methodological*. First, the simulator dominated the result: on the *identical* netlist, ngspice reported 0.71 on circles while Cadence Spectre reported 0.95 — ngspice was materially mis-integrating the slow capacitor-charging (learning) dynamics. Second, our own earlier numbers were inflated ~ 0.1 by a too-small test set and best-of-many-blocks reporting. The circuit's learning *mechanism is sound* (it trains a linearly separable control task to 1.000 on every random seed), and the dominant early defect (weight railing from a common-mode mismatch) was found and fixed. The remaining, real obstacle is **robust convergence on hard nonlinear tasks**: the features are perfectly separable for every seed (ridge = 1.000), yet the local delta-style rule reaches that solution only for favorable random-feature geometries. We argue this is a delta-rule fixed-point problem and that equilibrium propagation is the principled fix. **The > 0.95-on-all-three goal is not yet met.**

1 Problem and constraints

The goal is > 0.95 test accuracy on **circles** (concentric annuli), **rings**, and **spirals**, subject to:

- **Inference and training are entirely in SPICE.** Weights are physical state (differential charge on capacitor pairs). The update $\dot{W} \propto \varepsilon \cdot a_h$ is performed by analog cells. *No behavioral elements* — no B-source computing a gradient, no numeric update in the controller.
- **The controller only cycles data**: it writes the piecewise-linear schedule that presents `(input,label)` pairs over time and supplies bias/clock voltages. It never computes a gradient or a weight value.

This is a continuous-time predictive-coding (PC) learner realized physically.

2 Architecture

Figure 1 shows the signal flow. The hidden layer is a *fixed random* analog feature map; only the readout is trained.

Cells (all pure transistor/R/C/V, LEVEL-1 MOSFETs). `gsyn`: source-degenerated Gilbert multiplier (synapse). `dneuron`: differential-pair neuron with common-mode load. `esub`: two-tail

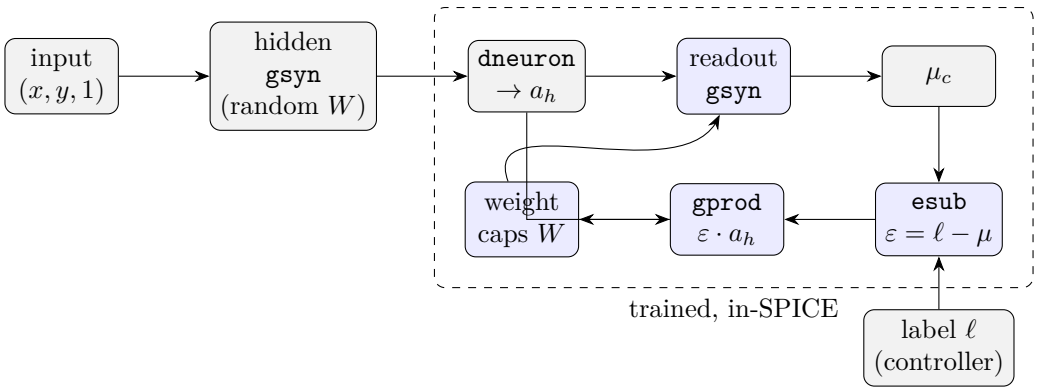

Figure 1: Fully in-SPICE PC learner. Fixed random hidden features (a_h) drive a trained readout whose weights are capacitor charge. `esub` forms the error $\varepsilon = \ell - \mu$; `gprod` (a four-quadrant Gilbert multiplier) charges the weight caps by the local product $\varepsilon \cdot a_h$. One `.tran` performs a clamped training phase then a frozen evaluation phase. All cells are transistor/R/C/V only.

differential subtractor forming $\varepsilon_c = \ell_c - \mu_c$. `gprod`: four-quadrant Gilbert product sourcing/sinking current into the weight caps $\propto \varepsilon_c \, a_{h,j}$. Signed weights are the differential charge $w = q^+ - q^-$; an in-circuit hinge gate (a soft comparator) can freeze a class's update once it is confidently correct. Transistor-level schematics of the three core cells (`gsyn`, `esub`, `gprod`) are given in Appendix A.

3 Two methodological findings

3.1 The simulator was the bottleneck

Running the *identical* netlist and configuration through two SPICE engines gives sharply different answers (Fig. 2). ngspice (LEVEL-1, `method=gear`, `reltol`=2e-3) reports circles = 0.713; Cadence Spectre (default trapezoidal, moderate `errpreset`) reports 0.950 on the same deck, in $\sim$5$\times$ less wall-clock time. The Spectre result is not an output-sampling artifact (strobed output 0.950 vs. full output 0.956). We conclude that a substantial part of what we had been calling an "in-circuit learning ceiling" was an **ngspice numerical artifact** in integrating the slow capacitor-charging dynamics. *Every prior ngspice-based negative conclusion must be re-checked on Spectre before being trusted.*

3.2 Our own numbers were inflated

For much of the project "circles $\approx$ 0.83" was reported. With a proper test set (NTE $\geq$ 80) and final-accuracy reporting, the true number was $\sim$ 0.5–0.73. The inflation came from (a) a 24-example test set (granularity 0.04, high variance) and (b) reporting the *best* over many interval-eval blocks. **Methodology rule:** NTE $\geq$ 80, report final accuracy (not best-of-blocks), average over $\geq$ 3 seeds.

4 Results

The central tension is shown in Fig. 3: the mechanism trains a separable task to 1.0 on every seed, and the circles features are *always* linearly separable (ridge = 1.000), yet the in-circuit learner reaches that solution only on favorable random-feature seeds. This is therefore a **learning-convergence failure**, not a feature or capacity failure. Moreover, no learning-rule knob rescues a bad seed (Fig. 4): gate on/off, target margin, and learning-current scale all leave seed 1 at 0.59–0.60.

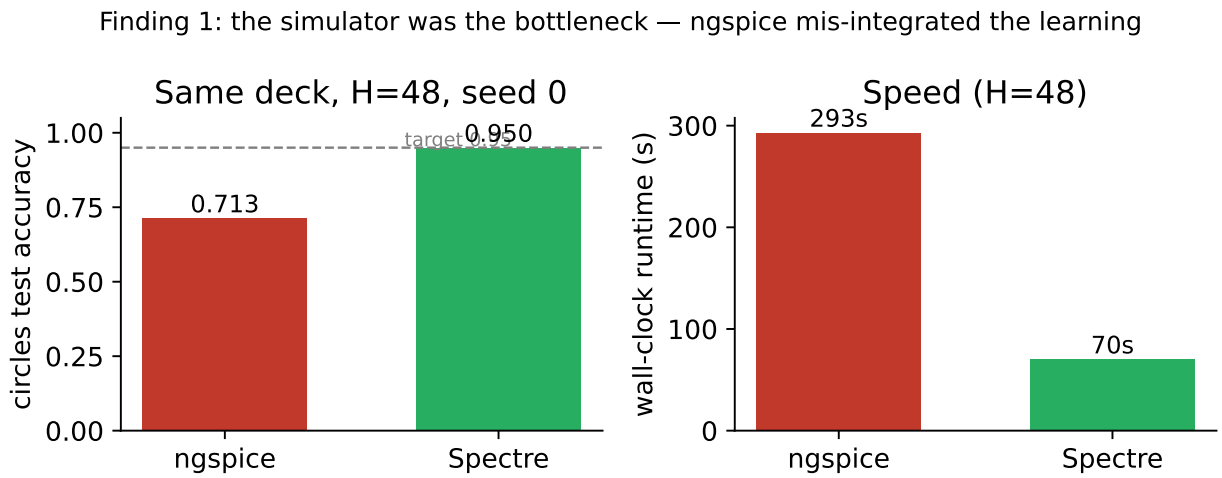

Figure 2: Same netlist, same configuration: ngspice vs. Spectre. The accuracy gap ($0.713 \rightarrow 0.950$) is a simulator effect, and Spectre is also faster (and completes H=160 in 5.5 min where ngspice times out).

Task	Setup	Result	Notes
blobs (linear control)	Spectre, H=48, all seeds	**1.00 / 1.00 / 1.00**	mechanism is sound
circles	Spectre, H=48, seed 0	**0.950–0.956**	meets target (one seed)
circles	Spectre, H=48, seeds 1,2,3	0.59 / 0.54 / 0.61	*not robust*
circles	ngspice, H=48, seed 0	0.713	same deck — simulator artifact
circles (capacity)	ridge on a_h, any seed	1.000	features always separable
rings, spirals	—	untested	not yet attempted

Table 1: Headline results. The goal (> 0.95 robustly on all three) is *not* met.

5 What works / what does not

Works (simulator-independent): source-degenerated Gilbert cells; the common-mode fix — the dominant early defect was an `esub` common-mode mismatch (readout μ-CM 0.666 vs. clamp 0.5) that railed the weights; a separate-tail `esub` plus a CM-pinned readout load collapsed railing $92\% \rightarrow 0\%$; the hinge gate (small real gain); and the Spectre `+mt` backend with `strobeperiod` output for fast, accurate iteration. The mechanism trains the linear control task to 1.0 on every seed.

Does not work (believed real, not ngspice noise): robust convergence on hard nonlinear tasks; learning-rule knobs do not rescue bad seeds; naive H-scaling is currently broken in the harness (H=120/200 collapse to 0.500; Spectre's `.save` does not limit nodes, producing 100 MB+ PSF files). A set of ideas that "failed" on ngspice (gradient-aware/replica feature, feature compression, attenuation, two-phase variants) are now *suspect* and need clean Spectre re-runs.

6 Related work

The relevant literature consistently avoids "raw SGD into a sloppy analog weight," favoring local rules, common-mode/reference cancellation, delayed transfer, and freeze/threshold logic. Closest to this work:

- **Equilibrium Propagation in nonlinear resistive networks** (Kendall et al.) — a SPICE/Spectre

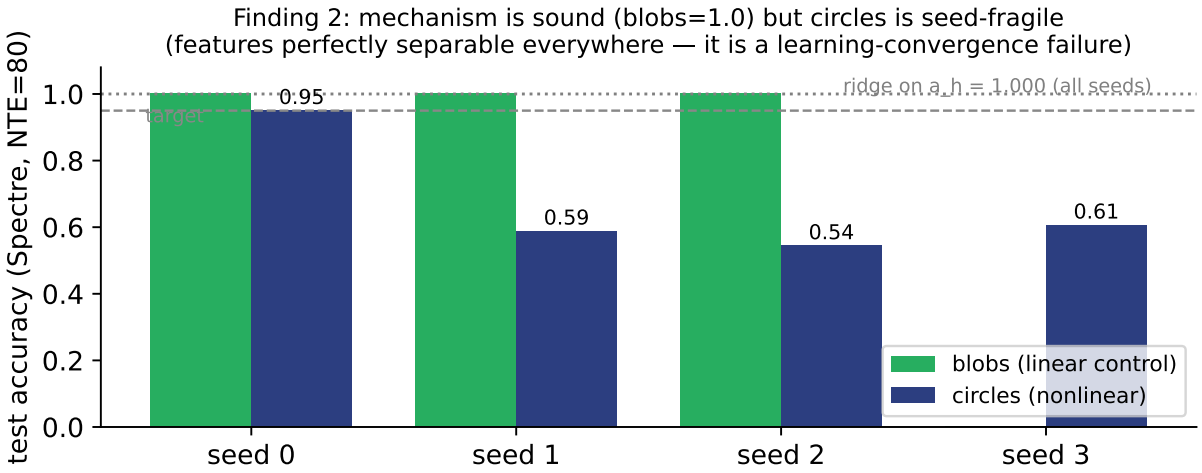

Figure 3: Mechanism sound, robustness not. blobs trains to 1.0 every seed; circles is seed-fragile despite ridge = 1.0 everywhere.

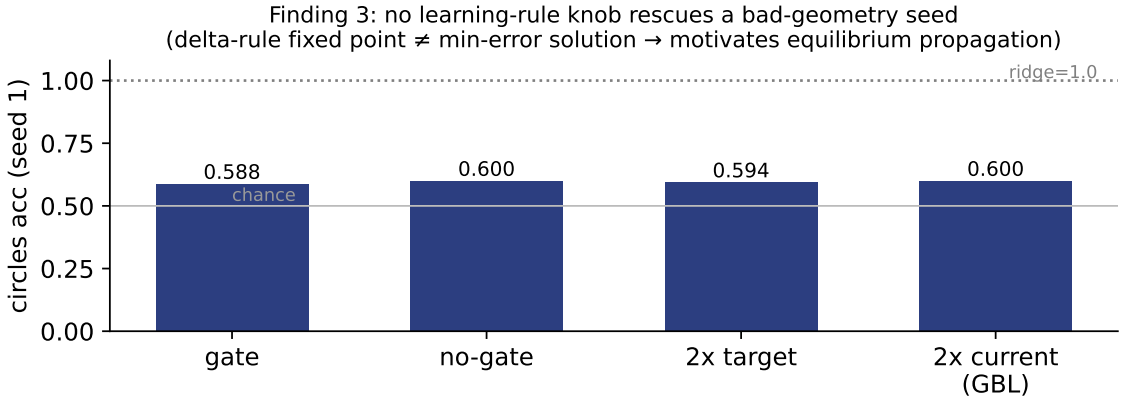

Figure 4: On a bad-geometry seed, none of the local learning-rule knobs move the result off ~ 0.59. This points at the *fixed point* of the rule, not its hyperparameters.

analog network with *local* learning where inference and training share the same physical circuit; MNIST with 100 hidden neurons, 3.43% test error. This is the most direct template, and it speaks precisely to our open problem: EP's update uses the same nonlinear circuit in a free and a nudged phase, so its stationary point *is* the (gradient-correct) min-error solution — exactly what a plain delta rule on a nonlinear readout lacks.

- **On-chip learning in a conventional MOSFET analog NN** (arXiv:1907.00625) — SPICE, ordinary MOSFETs (no memristors), small benchmark; closest in component philosophy.
- **Mixed-precision / FeCAP–memristor training** (Nature 2018; Nature Electronics 2025) — separate fast inference weights from higher-precision slow learning state; transfer every k samples.
- **IBM c-TTv2 / AGAD** (Nat. Commun. 2024) — normalize update magnitudes; *chopper*

/ *dynamic-reference* learning to cancel offsets (the algorithmic cousin of our common-mode fix).

- **Silicon-synapse memristive RBM/DBN** (arXiv:2203.09046) — binary neurons, ternary thresholded updates, blind writes.

Two themes from this literature map directly onto our results: (i) common-mode/reference cancellation is essential (we needed it to stop weight railing), and (ii) for nonlinear analog readouts, a *contrastive / two-phase* update (EP) is the principled way to make the learning fixed point coincide with the min-error solution.

7 Open problems and path forward

1. **Robustness is the crux.** Since ridge = 1.0 for all seeds, the target is reachable in principle; the analog *rule* must converge to it regardless of feature geometry. The leading candidate is a true **equilibrium-propagation** update (free + nudged phase, symmetric $\pm\beta$ nudge to cancel bias), now testable accurately on Spectre.
2. **Fix the H-scaling harness** (limit Spectre saved nodes; check convergence at large H) so "more features → robustness" can be tested cleanly.
3. **Re-audit ngspice-era negatives on Spectre** — several are likely false negatives.
4. **rings and spirals** have not been run in-SPICE at all and must clear the same bar.

8 Conclusion

A fully in-SPICE, no-behavioral-elements predictive-coding learner was built; its update is genuine analog charge driven by transistor multipliers, and its mechanism provably works (1.0 on a separable control task, every seed). The dominant early defect (weight railing from a common-mode mismatch) was diagnosed and fixed. The biggest lesson is methodological: ngspice was materially mis-simulating the learning, and an accurate simulator (Spectre) reaches 0.95 on circles where ngspice reported 0.71 — but only on favorable random-feature seeds. The remaining real obstacle is making the analog learning rule converge to the (provably reachable) solution *robustly*, for which equilibrium propagation is the principled next step. **The > 0.95-on-all-three goal is not yet met.**

A Cell schematics (transistor level)

All cells use LEVEL-1 NMOS/PMOS, source-degeneration resistors R, and a tail current source. Differential signals are used throughout ($\cdot_p, \cdot_n$). The synapse and the readout are the *same* `gsyn` cell (Fig. 5); the error neuron `esub` (Fig. 6) uses *separate tails* per input pair so the output depends on input *differences*, not common mode — the fix that stopped weight railing; the learning cell `gprod` (Fig. 7) converts the differential product current into capacitor charge.

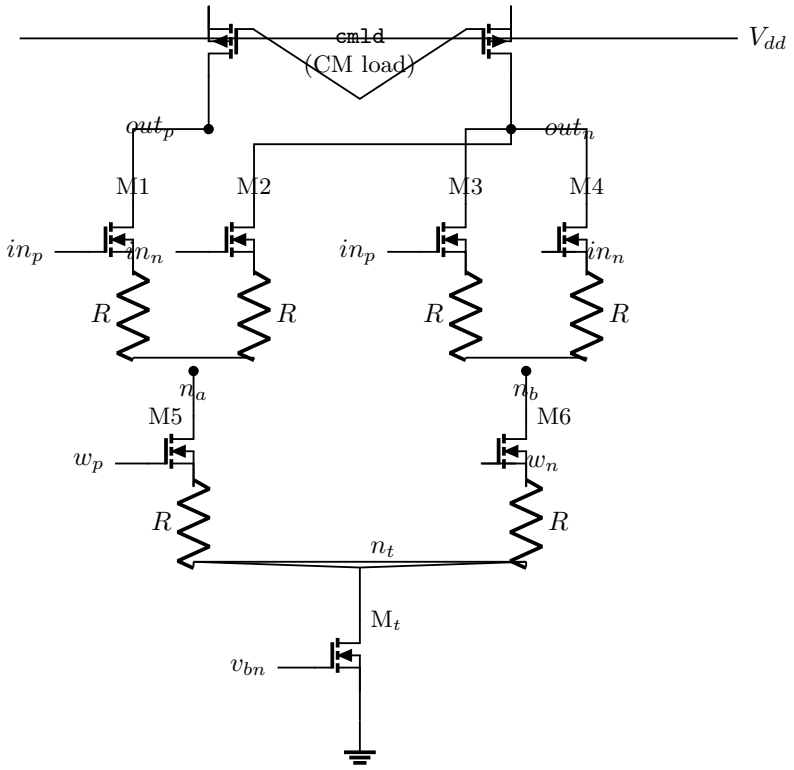

Figure 5: `gsyn` — source-degenerated Gilbert multiplier. The weight pair (M5,M6, gates w_p, w_n) steers the tail current; the input quad (M1–M4, gates in_p, in_n) forms the product at out_p, out_n. Degeneration resistors R linearize the cell. Used unchanged as both the hidden synapse and the trained readout.

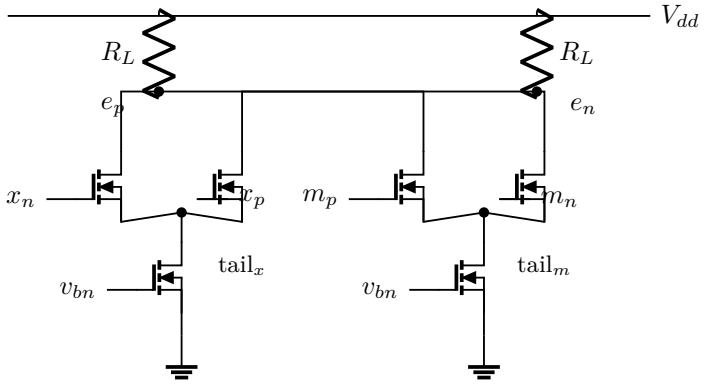

Figure 6: `esub` — separate-tail differential subtractor forming $\varepsilon = \ell - \mu$ at $e_p - e_n$. The label clamp (x_p, x_n) and the readout output (m_p, m_n) drive two diff pairs with *independent* tails into shared loads, so the output tracks the input *differences* and rejects common mode. Matching the clamp and readout common modes here is what collapsed weight railing $92\% \to 0\%$.

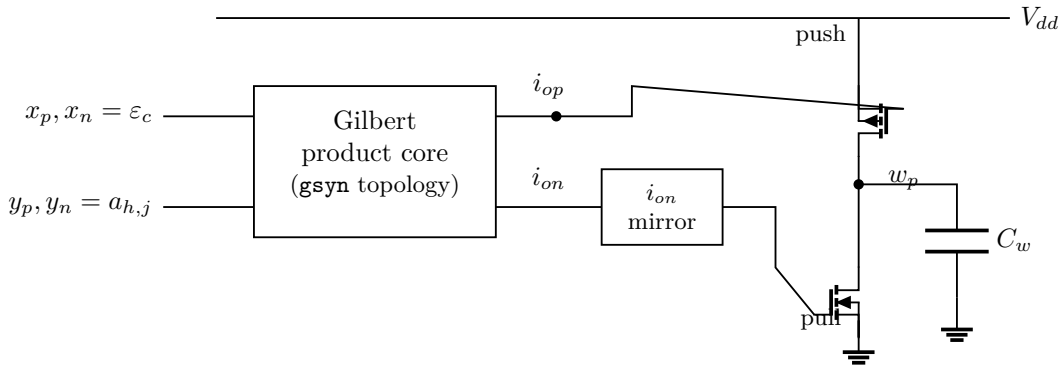

Figure 7: `gprod` — the learning cell. A Gilbert core (same topology as `gsyn`) forms the differential product current $i_{op} - i_{on} \propto \varepsilon_c\, a_{h,j}$; a push PMOS (gate = i_{op}) sources charge into the weight cap C_w while the mirrored i_{on} drives a pull NMOS that removes it, giving $\dot{w}_p \propto \varepsilon_c\, a_{h,j}$ — the predictive-coding update performed entirely by transistors. A mirror-symmetric cell drives w_n.